

Programação Orientada a Objetos - CST em Análise e Desenvolvimento de Sistemas

Exceções personalizadas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Motivação para o uso de exceções personalizadas

Não é obrigado a verificar o retorno de um método

- Podemos usar o retorno de um método para saber se ele executou com sucesso ou não
- Porém, esse projeto não obriga o programador a verificar o retorno

```
public boolean saque(double valor) {  
    if (this.saldo >= valor) {  
        this.saldo -= valor;  
        return true; // Sucesso  
    } else {  
        return false; // Saldo insuficiente  
    }  
}  
  
public static void main(String[] args) {  
    Conta conta = new Conta(100);  
    conta.saque(200); // Inovação de método sem verificar o retorno  
}
```

Motivação para o uso de exceções personalizadas

Diferentes tipos de retorno

- Trocando de boolean para int para retornar diferentes tipos de erro
- O programador precisa verificar o retorno e entender o que cada valor significa¹

```
public int saque(double valor) {  
    if (valor < 0){  
        return -1; // significa que o valor passado como parâmetro é negativo  
    }  
    if (this.saldo >= valor) {  
        this.saldo -= valor;  
        return 0; // Sucesso  
    } else {  
        return 1; // Saldo insuficiente  
    }  
}
```

¹Aqui seria melhor usar uma enumeração, mas vamos manter o exemplo simples para justificar um uso para exceções personalizadas

Problemas do uso de retornos para indicar erros

Não é possível indicar o motivo do erro

- Retorna nulo por que o CPF é nulo, inválido ou porque a pessoa não foi encontrada?
 - Três possíveis causas para um único valor de retorno

```
public Pessoa buscaPessoa(String cpf) {  
    if (cpf == null) {  
        return null;  
    }  
    if (!validaCPF(cpf)){  
        return null;  
    }  
    for (Pessoa p : pessoas) {  
        if (p.getCpf().equals(cpf)) {  
            return p;  
        }  
    }  
    return null;  
}
```

Uso de exceções no lugar de retornos

- **O uso de exceções obrigam o programador a tratar o evento fora do fluxo normal do programa**
 - O programador é forçado a tratar o erro, caso contrário o programa não compila
- **Classes de exceção do Java podem ser usadas para tratar situações específicas**
 - `IllegalArgumentException` para valores inválidos, `IllegalStateException` para estados inválidos, etc.
- **Classes de exceção personalizadas podem ser usadas para tratar problemas específicos**
 - Escreva uma classe que estenda `Exception` ou `RuntimeException`

Uso de exceções genéricas para tratar problemas específicos

```
public void saque(double valor) {  
    if (valor < 0){  
        throw new IllegalArgumentException("0 valor do saque não pode ser negativo");  
    }  
    if (this.saldo < valor) {  
        throw new IllegalStateException("Saldo insuficiente");  
    }  
    this.saldo -= valor;  
}  
  
public static void main(String[] args) {  
    Conta conta = new Conta(100);  
    try {  
        conta.saque(-200);  
    } catch (IllegalArgumentException e) { // Trata valor negativo  
        System.out.println("Erro: " + e.getMessage());  
    } catch (IllegalStateException e) { // Trata saldo insuficiente  
        System.out.println("Erro: " + e.getMessage());  
    }  
}
```

Classes personalizadas de exceção

- No exemplo anterior, usamos `IllegalArgumentException` e `IllegalStateException` para tratar problemas específicos
 - A exceção lançada é genérica e não expressa o problema de forma clara
- Uma classe de exceção personalizada pode ser mais expressiva, deixando claro a razão do erro
 - `SaldoInsuficienteException`
 - `ValorNegativoException`

Classes personalizadas de exceção

- No exemplo anterior, usamos `IllegalArgumentException` e `IllegalStateException` para tratar problemas específicos
 - A exceção lançada é genérica e não expressa o problema de forma clara
- Uma classe de exceção personalizada pode ser mais expressiva, deixando claro a razão do erro
 - `SaldoInsuficienteException`
 - `ValorNegativoException`

Dica

Geralmente é melhor usar as classes de exceções do Java, mas em alguns casos pode ser útil criar exceções personalizadas

Exemplo de exceção personalizada

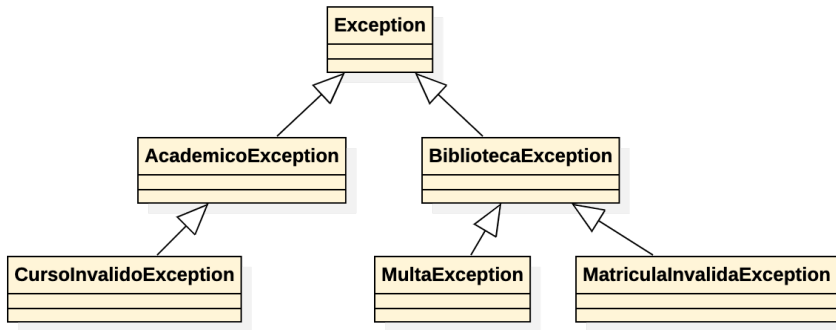
```
public class SaldoInsuficienteException extends Exception {  
    public SaldoInsuficienteException(String message) {  
        super(message);  
    }  
}
```

```
public void saque(double valor) throws SaldoInsuficienteException {  
    if (this.saldo < valor) {  
        throw new SaldoInsuficienteException("Saldo insuficiente");  
    }  
    this.saldo -= valor;  
}
```

```
public static void main(String[] args) {  
    Conta conta = new Conta(100);  
    try {  
        conta.saque(200);  
    } catch (SaldoInsuficienteException e) {  
        System.out.println("Erro: " + e.getMessage());  
    }  
}
```

Hierarquia de exceções de negócio

- Organizar as exceções por módulo de negócio (e.g. sistema acadêmico, biblioteca, etc.) pode facilitar a identificação do problema



Exercícios

Sistema de biblioteca

- Permitir que os usuários possam emprestar e devolver livros
- Lançar exceções personalizadas para tratar as seguintes situações:
 - Tentativa de empréstimo de um livro já emprestado
 - Tentativa de devolução de um livro que não foi emprestado
 - Limite de empréstimos atingido para um usuário (3 livros)

Implementação

- Criar classes de exceção personalizadas para cada situação, todas estendendo uma classe de exceção genérica `BibliotecaException`
- Implementar métodos para emprestar e devolver livros, lançando as exceções apropriadas quando necessário
- Criar uma classe com método *main* para testar o sistema, tratando as exceções lançadas pelos métodos de empréstimo e devolução